SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 50**

Question Count: 5

Code of Conduct

I **AMARA SATWIKA / 192525204** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **A.SATWIKA**

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

HDFS-Based Storage Solution

Answer:

For a media platform, I would design an HDFS storage system with a replication factor of 3 to provide high availability while staying within the given constraint.

Design:

- Store media files in HDFS using large block sizes.
- Set replication factor = 3.
- Place replicas on different DataNodes and preferably different racks.
- Use NameNode High Availability with an Active and Standby NameNode.
- Use automatic failover to maintain availability above 99.9%.
- Add DataNodes as storage requirements increase.
- Use compression for suitable media metadata and logs.
- Use data lifecycle policies to move old or less frequently accessed data to cheaper storage.

Justification:

Replication factor 3 provides fault tolerance if a DataNode fails. NameNode HA prevents a single point of failure. Horizontal scaling allows storage to grow without replacing the entire system. Compression and lifecycle management reduce storage costs.

Therefore, the design achieves >99.9% availability, keeps replication within 3, and provides cost-efficient storage growth**.**

2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

MapReduce Log Processing Within 20 Minutes

Answer:

The MapReduce job should divide the log data into multiple input splits and process them in parallel.

Job Design:

Input → Mapper → Combiner → Shuffle & Sort → Reducer → Output

Mapper:

- Reads individual log records.
- Extracts important information such as IP address, status code, timestamp, or URL.
- Produces key-value pairs.
- Example:
- (HTTP_Status, 1)

Combiner:

- The Combiner performs local aggregation before data is transferred to reducers.
- Example:
- (200, 500) instead of sending 500 individual records.

Reducer:

- Reducers aggregate the values received from multiple mappers.

Example:

(200, 5000)

Task Distribution:

- Divide the log file into multiple blocks.
- Assign each block to a Mapper.
- Run Mappers in parallel on available nodes.
- Distribute intermediate data among Reducers using the partitioner.
- Use a suitable number of reducers according to the limited cluster size.
- Prefer **data locality** so processing occurs near the stored data.

Meeting the 20-minute constraint:

- Use parallel Mapper execution.
- Use Combiner to reduce network traffic.

- Optimize input splits.
- Avoid unnecessary MapReduce stages.
- Allocate available nodes efficiently.

Thus, parallel processing, data locality, and intermediate-data reduction help complete the batch within the 20-minute window.

3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

YARN Scheduler Policy

Answer:

For a YARN cluster running ETL, reporting, and machine-learning jobs, I would use a Capacity Scheduler with priorities.

Proposed Policy:

Job	Priority	Resource Allocation
ETL	High	Highest
Reporting	Medium	Moderate
Machine Learning	Low/Medium	Remaining resources

Design:

- Create separate YARN queues for ETL, Reporting, and ML.
- Give ETL jobs higher priority because they are time-sensitive.
- Reserve minimum resources for reporting and ML so they are not completely blocked.
- Allow unused resources to be temporarily borrowed by other queues.
- Configure resource limits to prevent a single job from consuming the entire cluster.
- Enable YARN's automatic container restart when a task fails.
- Use NodeManager health monitoring to detect failed nodes.
- Reschedule failed tasks on healthy nodes.

Justification:

ETL receives priority while reporting and ML continue to receive resources. YARN's resource management and task recovery provide fault tolerance when a node fails.

Therefore, the policy provides fairness, ETL priority, efficient resource utilization, and recovery from node failures.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

SAN, NAS and Database Data Ingestion

Answer:

The enterprise can use a centralized Hadoop ingestion architecture.

Architecture:

SAN/NAS → Apache NiFi → HDFS

Operational Databases → NiFi/JDBC → HDFS

HDFS → Processing → Analytics

Components:

1. Apache NiFi

- Collects data from different sources.
- Supports database and file-system ingestion.
- Provides data routing and transformation.
- Maintains data-flow tracking.

2. HDFS

- Acts as the central storage layer.
- Uses replication for fault tolerance.
- Stores large volumes of structured and unstructured data.

3. Database Connector/JDBC

- Extracts data from operational databases.
- Supports incremental extraction to avoid repeatedly copying the same records.

4. Backup

- Maintain replicated HDFS copies.
- Use checkpoints and backup policies for critical data.
- Store important backup copies in separate storage when required.

Minimizing Duplication:

- Use unique record IDs.
- Perform incremental ingestion.
- Maintain ingestion metadata.
- Check whether a record has already been processed before loading it.
- Use deduplication during ETL processing.

Justification:

NiFi provides controlled ingestion from different sources, while HDFS provides scalable and fault-tolerant storage.

Incremental ingestion and unique identifiers reduce unnecessary duplication.

5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

End-to-End Hadoop Ecosystem Solution

The proposed solution integrates Apache NiFi, HDFS, YARN, MapReduce, and security mechanisms.

Architecture:

Data Sources

↓

Apache NiFi

↓

Secure Data Transfer

↓

HDFS

↓

YARN

↓

MapReduce / ETL Processing

↓

Analytics / Reporting

1. Secure Data Movement

- Use SSL/TLS for data transmission.
- Use authentication and authorization.
- Apply access permissions to HDFS directories.
- Encrypt sensitive data where required.

2. High Availability

- Use NameNode High Availability with Active and Standby NameNodes.
- Use HDFS replication.
- Distribute DataNodes across racks.
- Enable automatic failover.

3. Apache NiFi Integration

- NiFi collects data from databases, files, applications, and other sources.
- It can:

- Ingest data.
- Route data.
- Transform data.
- Monitor data flows.
- Send processed data to HDFS.

4. Hadoop Processing

- HDFS stores large datasets.
- YARN manages cluster resources.
- MapReduce processes large datasets in parallel.

5. Fault Recovery

- If a DataNode fails, HDFS uses other replicas. If a processing task fails, YARN can restart or reschedule the task on another available node.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured